

```

import time

def matrix_chain_order(p, n):

    dp = [[0] * n for _ in range(n)]

    for length in range(2, n):
        for i in range(1, n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')

            for k in range(i, j):
                cost = (
                    dp[i][k]
                    + dp[k + 1][j]
                    + p[i - 1] * p[k] * p[j]
                )

                if cost < dp[i][j]:
                    dp[i][j] = cost

    return dp[1][n - 1]

# ----- USER INPUT -----

num = int(input("Enter number of matrices: "))

print("Enter dimensions:")
print("For matrices A1, A2, ..., An, enter n+1 dimensions.")

p = []

for i in range(num + 1):
    value = int(input(f"Enter dimension p[{i}]: "))
    p.append(value)

# ----- EXECUTION TIME -----

start_time = time.perf_counter()

minimum_cost = matrix_chain_order(p, num + 1)

end_time = time.perf_counter()

execution_time = end_time - start_time

# ----- OUTPUT -----

print("\nMinimum number of scalar multiplications:",
      minimum_cost)

print("Execution time: {:.9f} seconds".format(execution_time))

print("\nTime Complexity: O(n^3)")
print("Space Complexity: O(n^2)")

```

Enter number of matrices: 9
Enter dimensions:
For matrices A1, A2, ..., An, enter n+1 dimensions.
Enter dimension p[0]: 1
Enter dimension p[1]: 4
Enter dimension p[2]: 8
Enter dimension p[3]: 7
Enter dimension p[4]: 9
Enter dimension p[5]: 5
Enter dimension p[6]: 8
Enter dimension p[7]: 4
Enter dimension p[8]: 5
Enter dimension p[9]: 9

Minimum number of scalar multiplications: 333
Execution time: 0.000276664 seconds

Time Complexity: $O(n^3)$
Space Complexity: $O(n^2)$